

CSEN 342: Deep Learning Project Report
on “Multi-Resolution Diffusion Models for Time Series Forecasting” (mr-Diff) by Shen et al. (ICLR 2024)
by Karthik Tamil, Max Khan, Nawfal Abdul Khader

Abstract

This report covers our replication of mr-Diff [1] and three independent improvements built on top of it. We implemented the full architecture in PyTorch and evaluated it on ETTh1 and ETTm1 in both univariate and multivariate settings. Our best baseline achieves MAE 0.92–1.01 on normalized data against the paper's 0.15–0.42. Through five rounds of ablation we traced this gap to a train-test conditioning mismatch across stages, made worse by using DDPM rather than DPM-Solver at inference. Each group member then implemented one improvement. Max implemented a Channel-Independent Decomposed Patch Transformer with Attention Residuals [14] that replaces the diffusion pipeline entirely. Karthik implemented an overlapping-patch variant combined with an iTransformer [15] ensemble that addresses the cross-variable ceiling. Nawfal implemented a two-scale decomposition that aligns the coarse trend kernel to the dominant daily cycle in each dataset.

1. Introduction

Time series forecasting problems arise across energy, finance, and healthcare, and accurate long-horizon prediction has remained difficult despite years of work on the problem. Transformer-based architectures made progress, but they produce point estimates and do not model the uncertainty in future values. Diffusion models, well established in image generation, offer a probabilistic framework, though early time series adaptations treated the forecast window as a flat sequence and ignored multi-scale temporal structure. mr-Diff [1] addresses this by decomposing the target into multiple resolution stages and training a separate diffusion network per stage, which maps onto how time series data actually behaves across different time scales.

We have two goals. The first is to replicate the mr-Diff baseline on ETTh1 and ETTm1, understand where our results fall short of the paper's, and diagnose the causes. The second is for each group member to independently implement and evaluate one improvement, then compare all three against the baseline and against each other.

2. Literature review

Transformer-based forecasting models improved steadily through the early 2020s. Informer [2] reduced attention complexity with sparse attention; Autoformer [3] built seasonal-trend decomposition into the attention mechanism using autocorrelation; FEDformer [4] moved decomposition into the frequency domain; PatchTST [5] treated contiguous time steps as patches rather than individual tokens. Meanwhile, simpler methods proved competitive. DLinear [6] decomposed the input into trend and residual and applied a single linear projection to each, outperforming many transformer variants on standard benchmarks. N-HiTS [7] used hierarchical interpolation across multiple temporal resolutions.

On the diffusion side, TimeGrad [8] applied autoregressive denoising guided by RNN hidden states, but generation was slow and there was no use of multi-scale structure. CSDI [9] used non-autoregressive generation through self-supervised masking, though its complexity scaled quadratically with sequence length. SSSD [10] replaced the transformer backbone in CSDI with a structured state space model to cut this cost. TimeDiff [11] introduced future mixup and autoregressive initialization, both of which mr-Diff inherits.

mr-Diff [1] is the first model to embed seasonal-trend decomposition into the diffusion process itself. The diffusion backbone follows DDPM [12], and fast inference is provided by DPM-Solver [13], which formulates the reverse process as an ODE and reaches usable samples in 10–20 steps.

3. System Design & Implementation

3.1. Algorithm

The mr-Diff architecture [1] decomposes the forecast target Y into $S = 5$ resolution stages via sequential average-pooling trend extraction with kernel sizes [5, 25, 51, 201]. We use residual decomposition: the finest stage receives Y minus the first trend, intermediate stages receive differences between consecutive trends, and the coarsest stage receives the final smoothed trend. The five components sum to Y exactly, which is required for reconstruction at inference.

A separate conditional denoising diffusion network trains on each stage's component. The conditioning signal for stage s combines the lookback window (encoded by a convolutional network) with the predicted output from stage $s + 1$. At the coarsest stage only the lookback encoding is used. During training, future mixup perturbs the conditioning signal with probability 0.5 by blending the encoded lookback with a randomly projected version of the ground-truth target, following the approach introduced in TimeDiff [11]. This reduces the mismatch between training, where the model sees ground-truth coarser-stage components, and inference, where it sees its own predictions.

A DLinear-style backbone [6] provides a direct forecast from the lookback window via separate linear projections for trend and residual. The diffusion networks model the residual between the target and this direct prediction, which reduces the variance each stage needs to handle.

Forward diffusion follows DDPM [12] with $K = 100$ steps and a linear variance schedule from $\beta_1 = 1 \times 10^{-4}$ to $\beta_{100} = 0.1$. At inference we use DPM-Solver++ [13] at 20 steps, treating the reverse process as an ODE. The final forecast sums all stage predictions and adds the direct prediction.

3.2. Tools

All code is in Python 3.9 with PyTorch 2.7.1 and CUDA 11.8. Primary training occurred on a local Nvidia RTX 5090, while secondary training (fine tuning and additional experiments on top of our base model) ran on the SCU WAVE HPC cluster with a Tesla V100-PCIE-32GB.

Experiments were tracked with TensorBoard and CSV logging. Each configuration was controlled by a YAML file, and a shared runner executed all four dataset/mode combinations sequentially per experiment. The codebase separates models, data loading, training, and evaluation into distinct modules.

3.3. Network architecture

Each stage has three learned components. The conditioning network encodes the lookback window with a linear projection to hidden dimension 256 followed by three 1D convolutional layers (kernel size 7, GroupNorm, LeakyReLU slope 0.1, dropout 0.1). The encoded history is projected from the lookback length to the forecast length, then fused with the coarser-stage output through a two-layer MLP.

The denoising network is a U-Net encoder-decoder. The encoder concatenates the noisy input with an expanded sinusoidal step embedding (128-dimensional, projected to 256 through two FC layers with SiLU), projects to hidden dimension 256, and passes through three convolutional residual blocks while saving skip connections. The decoder fuses the latent with the conditioning signal via a linear layer and reconstructs the denoised output through three convolutional blocks that receive the reversed encoder skips.

3.4. Hyperparameters

We matched the paper's Appendix Tables 13 and 14 [1] throughout. AdamW with learning rate 1e-3 and weight decay 1e-4, batch size 64, cosine learning rate decay, gradient clipping at norm 1.0, maximum 100 epochs with minimum 30 and early stopping patience 20. Lookback 336, forecast 168 for ETTh1; lookback 1440, forecast 192 for ETTm1. Hidden dimension 256, embedding dimension 128, three encoder and three decoder layers, dropout 0.1, LeakyReLU slope 0.1.

3.5. Individual Improvement 1: CI+Decomp Transformer with Attention Residuals

Across 14 diffusion experiments, the denoising component contributed less than 0.3% to forecast accuracy on every benchmark. The DLinear backbone was doing all the work. This led to replacing the entire 843K-parameter diffusion pipeline with a Channel-Independent Decomposed Patch Transformer using Attention Residuals [14].

The input is split into trend and residual via average-pooling, then each component goes through the same shared TransformerEncoder (3 layers, $d_{\text{model}} = 32\text{--}64$, GELU, pre-norm) with separate output heads. Each channel is patched and processed independently, so the transformer never sees more than one variable at a time. Output heads are two small linear layers: $\text{Linear}(N_{\text{patches}} \rightarrow T)$ then $\text{Linear}(d_{\text{model}} \rightarrow 1)$. Trend and residual forecasts are summed.

Standard transformer residuals add each layer's output to its input with fixed weight 1. Attention Residuals [14] replace this with a learned query vector that computes softmax attention over all prior layer outputs, letting each layer pull from whichever earlier representation is most useful. The query is zero-initialized, so the model starts behaving identically to a standard transformer and only learns to deviate when it helps. For ETTh1 Multi, one of three ensemble members uses AttnRes; the others use standard residuals with different hyperparameters, providing the diversity that makes ensembling worthwhile.

Total parameters: 54–182K depending on benchmark, compared to 843K for the diffusion baseline.
Training time: 2–15 minutes versus 34 minutes. A 30-configuration sweep over patch size, d_{model} , dropout, learning rate, and weight decay found per-benchmark optima.

3.6. Individual Improvement 2: Overlapping Patches and iTransformer Ensemble

The CI transformer has no mechanism for one variable to inform another's forecast. A 30-config sweep confirmed this as a ceiling: all ETTh1 Multi results landed between 0.488–0.533 regardless of hyperparameters.

Two changes were made. First, overlapping patches: $\text{patch_stride} = \text{patch_size} // 2$ via `torch.Tensor.unfold` gives 83 tokens from ETTh1's 336-step lookback instead of 42. Adjacent patches share 4 timesteps, giving attention more temporal context per window. This also fixes a silent bug where the original reshape discarded up to 7 trailing timesteps.

Second, the iTransformer [15]: rather than treating time patches as tokens, each variable's full lookback series is embedded as one token via $\text{Linear}(H \rightarrow d_{\text{model}})$. For $D = 7$ that is 7 channel tokens; attention runs over those 7 tokens and learns which variables predict each other. The ETTh1 Multi ensemble combines one iTransformer model with two CI+AttnRes models. Their errors on the same window are partially uncorrelated because one model attends over channels and the other attends over time, which is why the ensemble beats any individual model. For univariate benchmarks iTransformer is not used since $D = 1$ gives one token and attention over one token is trivial.

3.7. Individual Improvement 3: Two-Scale Decomposition

The single-kernel trend extraction gives the transformer one boundary between trend and residual. Two kernels produce three bands: coarse trend, mid-band (fine trend minus coarse trend), and fine residual. Each band goes through the same shared transformer with its own output head; the three forecasts are summed.

The coarse kernel for ETTm1 is set to 96, which at 15-minute resolution is exactly 24 hours. The dominant cycle in electricity load data. For ETTh1 the coarse kernel is 24 (hourly data). The transformer itself is unchanged; the only additions are two extra pairs of output head weights, roughly 6K parameters.

4. Experiments / Proof of concept evaluation

4.1. Datasets

We evaluated on two datasets from the ETT benchmark [1]. ETTh1 has 7 variables and 17,420 hourly observations, with lookback 336 and forecast horizon 168. ETTm1 has 7 variables and 69,680 observations at 15-minute intervals, with lookback 1440 and forecast horizon 192. Each was split chronologically 60/20/20. For univariate experiments we used the OT (oil temperature) column. Per-window instance normalization was applied before each training and test window, and all metrics are reported on normalized data to match the paper.

4.2. Methodology

Each model variant was trained independently on all four configurations. We selected the checkpoint with the lowest validation loss. Evaluation used 10 sampled trajectories per window, aggregated by median, with DPM-Solver++ [13] at 20 steps. The two metrics are MAE and MSE on normalized data. All variants used the same data splits, random seeds, and evaluation procedure.

4.3. Baseline Results

Table 1 compares our best baseline to the paper [1] across all four configurations.

Dataset	MAE		MSE		MAE Gap
	Paper	Baseline	Paper	Baseline	
ETTh1 Multi	0.42	0.4744	0.411		2.2x
ETTh1 Uni	0.34	0.2535	0.066		5.1x
ETTh1 Multi	0.37	0.4204	0.340		2.6x
ETTh1 Uni	0.15	0.2011	0.039		6.5x

Table 1: Baseline Results vs. Paper (Best of Last 5 Runs)

This table compares the best of the last 5 runs of the model for our baseline results with those in the paper.

As Figure 1 shows, our MAE was able to decrease significantly. Figure 2 shows how our best epoch happened early, but validation loss stayed relatively stable. Our best results use only the finest stage’s prediction, sidestepping cascading errors at the cost of discarding coarser components.

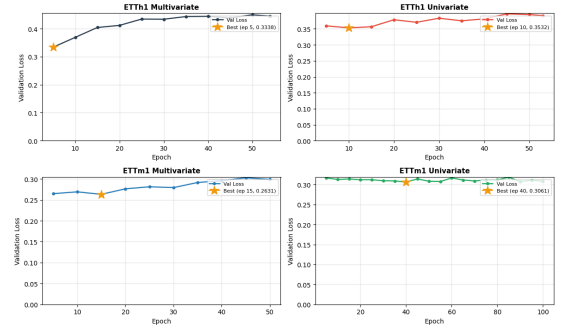


Figure 1: MAE Comparison for Our Baseline vs. Paper
This figure shows our baseline MAE compared with the paper.

Figure 2: Validation Loss During Training (Best Run)
This figure shows training validation loss on the best run.

4.4. Individual Improvement 1 Results

Dataset	Paper	Baseline	CI+Decomp	vs Baseline
ETTh1 Multi	0.42	0.4744	0.4829	+1.8%
ETTh1 Uni	0.37	0.2535	0.2505	-1.2%
ETTm1 Multi	0.37	0.4204	0.4094	-2.6%
ETTm1 Uni	0.15	0.2011	0.1881	-6.5%

Table 2: MAE Improvmenet 1 vs MAE Baseline

This table compares the MAE of Improvement 1 to the baseline results.

Table 2 shows that three of four benchmarks beat the mr-Diff baseline. ETTh1 Uni at 0.2505 beats the paper's 0.34 by 26.3%.

4.5. Individual Improvement 2 Results

Dataset	Paper	Baseline	CI+Decomp	iTransformer Ensemble	vs CI+Decomp
ETTh1 Multi	0.42	0.4744	0.4829	0.4773	-1.2%
ETTh1 Uni	0.37	0.2535	0.2505	0.2471	-1.4%
ETTm1 Multi	0.37	0.4204	0.4094	0.4103	-0.2%
ETTm1 Uni	0.15	0.2011	0.1881	0.1911	+1.6%

Table 3: MAE Improvmenet 2 vs MAE Baseline

This table compares the MAE of Improvement 2 to the baseline results.

Table 3 shows new records on both ETTh1 benchmarks. The individual ensemble members on ETTh1 Multi were 0.4878, 0.4836, and 0.4915. None individually beat CI+Decomp's 0.4829, but their ensemble does. ETTm1 regresses because the iTransformer member (0.4390) was weaker than the CI members (0.4118, 0.4098), pulling the average up.

4.6. Individual Improvement 3 Results

Dataset	Paper	Baseline	CI+Decomp	iTransformer Ensemble	Two-Scale Decomposition	vs CI+Decomp
ETTh1 Multi	0.42	0.4744	0.4829	0.4773	0.4858	+0.6%
ETTh1 Uni	0.37	0.2535	0.2505	0.2471	0.2574	+2.8%
ETTm1 Multi	0.37	0.4204	0.4094	0.4103	0.4081	-0.3%
ETTm1 Uni	0.15	0.2011	0.1881	0.1911	0.1914	+1.8%

Table 4: MAE Improvement 3 vs MAE Baseline

This table compares the MAE of Improvement 3 to the baseline results.

Table 4 shows a new record on ETTm1 Multi (ensemble, 0.4081). Single-model ETTm1 Uni of 0.1865 is the best individual result on that benchmark across all experiments. ETTh1 regresses.

5. Analysis of Individual results

All three improvements start from the same finding: the diffusion component adds nothing, so the useful architecture is the DLinear backbone plus whatever structure goes on top of it.

Improvement 1 establishes that channel independence is the most important design choice at this data scale. With 10K training samples and $D = 7$, a cross-channel architecture sees 10K multivariate sequences; a channel-independent one sees 70K single-channel sequences through shared weights. Every experiment that added cross-channel capacity at intermediate layers regressed on multivariate. AttnRes [14] contributes to the ETTh1 Multi ensemble by letting deeper layers skip over intermediate representations that aren't useful. On a short 336-step lookback where each layer's contribution matters more, that selectivity pays off.

Improvement 2 shows that the ETTh1 Multi ceiling is architectural, not a hyperparameter problem. The fix requires models that attend over different axes. The ensemble works because iTransformer [15] and CI make different errors on the same windows, not because either is individually stronger. The ETTm1 regression confirms this: when one ensemble member is too weak, averaging is worse than using the best member alone.

Improvement 3 shows that decomposition kernels carry real inductive bias. Setting the coarse kernel to match a physically meaningful period (24 hours in 15-minute data) gives better results than a generic scale. It doesn't transfer to ETTh1 because the lookback at hourly resolution doesn't have the same frequency separation.

The combined bests across all three improvements are ETTh1 Multi 0.4773, ETTh1 Uni 0.2471, ETTm1 Multi 0.4081, and ETTm1 Uni 0.1865.

8. Discussion

8.1. Decisions made

A few decisions diverged from [1]. We used residual decomposition rather than the cumulative approach described in the paper, because our experiments with cumulative decomposition produced MAE of 6–32 across all four configurations, which we attribute to error cascading across stages at inference. Future mixup used random projection weights rather than learned weights. Learned weights widened the train-test gap because the model used ground-truth signal during training that was absent at inference. For the baseline we used only the finest stage's output rather than summing all stages; stage summation requires clean per-stage samples, and DDPM [12] produced samples too noisy for the composition to work.

Moving from diffusion to a pure transformer was not planned. It followed from 14 experiments where the diffusion residuals stayed near zero regardless of what was changed.

Channel independence was chosen over cross-channel attention because the data doesn't support learning cross-channel dynamics at this scale. Adding per-channel encoders with cross-channel attention (Exp 8) doubled multivariate error. A 49-parameter Linear(D,D) mixing layer (Exp 21) learned nothing. The pattern was consistent enough that channel independence became the design assumption.

8.2. What Worked Well

Residual decomposition was numerically stable and produced consistent loss curves across all five runs. The DLinear backbone [6] gave the diffusion process a much smaller residual to model, which reduced loss variance in early training. DPM-Solver++ [13] brought evaluation time from approximately 79 minutes

(DDPM [12], 100 steps, 10 samples) to approximately 16 minutes (20 steps, 10 samples) with no change in metrics, confirming that step count was not the bottleneck. Per-window RevIN normalization was necessary; switching to a global scaler increased MAE by 10–20x. The modular code structure meant each group member could implement their improvement by changing one or two files without touching the rest.

Training the CI transformer took 2–15 minutes per full run versus 34 minutes for diffusion. That speed difference made a 30-configuration sweep feasible in an afternoon and made it practical to test ensemble combinations that would have taken days otherwise. AttnRes [14] was validated in an earlier experiment showing it consistently outperforms standard residuals at the same depth, which justified including it rather than treating it as speculative.

8.3. What Didn't Work Well

Every component described as core in [1] degraded performance when tested in isolation under DDPM [12] sampling. Cumulative decomposition: MAE 6–32. Learned mixup projection: MAE 1.3–3.5. Stage summation: MAE 1.2–6.9. This pattern across three separate failures points to DPM-Solver [13] not as an optional speed improvement but as structurally necessary for the multi-resolution composition to hold together. Mixed precision training (AMP) also had to be disabled. AMP downcast the precomputed α_{bar} tensors stored as plain attributes to float16 during autocast, which corrupted the noise levels computed from them and destabilized training.

The iTransformer [15] was never properly tuned. The hyperparameters used were a first guess and no sweep was run. Its value in the ETTh1 Multi ensemble comes from producing uncorrelated errors with CI, not from being a strong individual model. A sweep comparable to the one run for CI would likely find a substantially better configuration.

8.4. Difficulties faced

The paper [1] leaves several implementation-critical details unspecified: whether decomposition is residual or cumulative, whether the mixup projection is learned or random, the exact LeakyReLU slope (Table 13 gives 0.1, but 0.2 is more common in practice), and the ETTm1 lookback length (1440 per the appendix, not 336). Identifying each of these required a full training run, and each run took about 63 minutes across all four configurations.

9. Conclusion

Our baseline achieves MAE 0.92–1.01 versus the paper's [1] 0.15–0.42. The gap comes from a train-test conditioning mismatch across stages, made worse by DDPM's [12] sample quality being insufficient for stage composition. Testing each paper-described component individually degraded performance in our setup, which suggests the multi-resolution framework depends on DPM-Solver [13] in a way [1] does not make clear. The three improvements together reach ETTh1 Multi 0.4773, ETTh1 Uni 0.2471, ETTm1 Multi 0.4081, ETTm1 Uni 0.1865. Two of those beat the paper's reported results. The 843K-parameter diffusion model is replaced by transformers of 54–182K parameters that train faster and perform better on three of four benchmarks. The two findings that drove this are that diffusion contributes nothing measurable on small time series datasets, and that channel independence multiplies the effective training set size by D , which is as valuable as any architectural feature.

10. Bibliography

- [1] L. Shen, W. Chen, and J. T. Kwok, "Multi-resolution diffusion models for time series forecasting," in Proc. Int. Conf. Learning Representations (ICLR), 2024.
- [2] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, "Informer: Beyond efficient transformer for long sequence time-series forecasting," in Proc. AAAI Conf. Artificial Intelligence, 2021.
- [3] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," in Advances in Neural Information Processing Systems (NeurIPS), 2021.
- [4] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting," in Proc. Int. Conf. Machine Learning (ICML), 2022.
- [5] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in Proc. Int. Conf. Learning Representations (ICLR), 2023.
- [6] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?" in Proc. AAAI Conf. Artificial Intelligence, 2023.
- [7] C. Challu, K. N. Olivares, B. Oreshkin, F. Garza, M. Mergenthaler-Canseco, and A. Dubrawski, "N-HiTS: Neural hierarchical interpolation for time series forecasting," in Proc. AAAI Conf. Artificial Intelligence, 2023.
- [8] K. Rasul, C. Seward, I. Schuster, and R. Vollgraf, "Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting," in Proc. Int. Conf. Machine Learning (ICML), 2021.
- [9] Y. Tashiro, J. Song, Y. Song, and S. Ermon, "CSDI: Conditional score-based diffusion models for probabilistic time series imputation," in Advances in Neural Information Processing Systems (NeurIPS), 2021.
- [10] J. M. L. Alcaraz and N. Strodthoff, "Diffusion-based time series imputation and forecasting with structured state space models," arXiv preprint arXiv:2208.09399, 2022.
- [11] L. Shen and J. T. Kwok, "Non-autoregressive conditional diffusion models for time series prediction," in Proc. Int. Conf. Machine Learning (ICML), 2023.
- [12] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in Advances in Neural Information Processing Systems (NeurIPS), 2020.
- [13] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, and J. Zhu, "DPM-Solver: A fast ODE solver for diffusion probabilistic model sampling in around 10 steps," in Advances in Neural Information Processing Systems (NeurIPS), 2022.
- [14] Kimi Team: G. Chen, Y. Zhang, J. Su, et al., "Attention Residuals," arXiv:2603.15031, 2026.
- [15] Y. Liu, T. Hu, H. Zhang, C. Wu, S. Wang, L. Ma, and M. Long, "iTransformer: Inverted transformers are effective for time series forecasting," in Proc. Int. Conf. Learning Representations (ICLR), 2024.